Assignment 01: Xv6 and Unix Utilities (util)

CSci 430: Introduction to Operating Systems

Objectives

This lab will familiarize you with xv6 and its system calls.

- Make sure you understand how to build and run xv6, and add new user programs to the running qemu xv6 emulation.
- Become familiar with xv6 system calls, like `pause()`, `open()` `read()` and how to use them.
- Practice using low-level C
  - Practice and be more comfortable with C pointers
  - Practice using pointer arithmetic
  - Practice and understand C character arrays and C string processing
- Learn about and use more complex xv6 system calls, like `fstat()`, `fork()` and `exec()`

Description

Have a look at the Getting Started instructions for information on how to set up your computer to work on and run these assignments.

Note: this should be discussed in class, but as this is the first assignment. Remember that assignments are submitted to GitHub classrooms by creating a commit and pushing it back to your assignment repository. You are required to make a commit for at least each task as you complete it (though you can make more than one commit for a task if needed). In general, from the command line, you can create and push a commit by doing something like:

```
$ git commit -am "My solution for Assignment 01 Task 02, sleep user program"
$ git push
```

Assignment Tasks

Task 01: Boot xv6 (easy)

You should have already accepted this assignment, cloned the repository, and reopened the local repository workspace in a docker / VSCode DevContainer.

If your docker container is set up and running correctly, you should have all of the gcc RISCV toolchain tools installed and available to build and run the xv6 code in these assignments.

Test that you can successfully build and run xv6:

```
$ make qemu
riscv64-linux-gnu-gcc -g -c -o kernel/entry.o kernel/entry.S
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -gdwarf-2
    -DSOL_UTIL -DLAB_UTIL -MD -mcmodel=medany -ffreestanding -fno-common -nostdlib
    -fno-builtin-strncpy -fno-builtin-strncmp -fno-builtin-strlen -fno-builtin-memset
    -fno-builtin-memmove -fno-builtin-memcmp -fno-builtin-log -fno-builtin-bzero
    -fno-builtin-strchr -fno-builtin-exit -fno-builtin-malloc -fno-builtin-putc
    -fno-builtin-free -fno-builtin-memcpy -Wno-main -fno-builtin-printf
    -fno-builtin-fprintf -fno-builtin-vprintf -I. -fno-stack-protector
    -fno-pie -no-pie   -c -o kernel/kalloc.o kernel/kalloc.c
```

```
...
riscv64-linux-gnu-ld -z max-page-size=4096 -T user/user.ld -o user/_memdump user/memdump.o
    user/ulib.o user/usys.o user/printf.o user/umalloc.o
riscv64-linux-gnu-objdump -S user/_memdump > user/memdump.asm
riscv64-linux-gnu-objdump -t user/_memdump | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d'
    > user/memdump.sym
mkfs/mkfs fs.img README user/findtest.sh user/sixfive.txt user/_cat
    user/_echo user/_forktest user/_grep user/_init user/_kill user/_ln user/_ls
    user/_mkdir user/_rm user/_sh user/_stressfs user/_usertests user/_grind
    user/_wc user/_zombie user/_logstress user/_forphan user/_dorphan  user/_memdump
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total 2000
balloc: first 942 blocks have been allocated
balloc: write bitmap block at sector 46
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3
    -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0
    -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$
```

If you type `ls` at the prompt, you should see output similar to the following:

```
$ ls
.              1 1 1024
..             1 1 1024
README         2 2 2425
findtest.sh    2 3 91
sixfive.txt    2 4 19
cat            2 5 35096
echo           2 6 33992
forktest       2 7 17048
grep           2 8 42456
init           2 9 34472
kill           2 10 33920
ln             2 11 33744
ls             2 12 41208
mkdir          2 13 33984
rm             2 14 33968
sh             2 15 55824
stressfs       2 16 34864
usertests      2 17 188408
grind          2 18 49968
wc             2 19 35976
zombie         2 20 33344
logstress      2 21 35880
forphan        2 22 34808
dorphan        2 23 34272
memdump        2 24 35320
console        3 25 0
```

These are the files that `mkfs` includes in the initial file system; most are programs you can run. You just ran one of them: `ls`. xv6 has no `ps` command (process status, e.g. list of running processes), but if you type `ctrl-p`, the kernel will print information about each process. If you try it now, you'll see two lines, one for `init`, and one for `sh`, which are the two processes that are running currently:

```
$ <ctrl-p>
1 sleep  init
2 sleep  sh
```

To quit qemu type `ctrl-a x` (press `ctrl` and `a` at the same time, followed by `x`. You can type `ctrl-a ?` for a list of other qemu meta-commands).

There is nothing yet to commit, but for all assignments for the class, you need to ensure you can build xv6 cleanly and run it in the qemu emulator before proceeding to the assignment tasks.

Task 02: `sleep` user program (easy)

This exercise makes you familiar with writing a user program on xv6 and the `pause()` system call.

Implement a user-level `sleep` program for xv6, along the lines of the UNIX `sleep` command. Your `sleep` should pause for a user-specified number of ticks. A tick is a notion of time defined by the xv6 kernel, namely the time between two interrupts from the timer chip. Your solution should be in the file `user/sleep.c`.

Some hints:

- Before you start coding, read Chapter 1 of the xv6 book.
- Put your code in the `user/sleep.c` file. Look at some of the other programs in `user/` (e.g. `user/echo.c`, `user/grep.c`, and `user/rm.c`) to see how command-line arguments are passed to a program.
- Add your sleep program to `UPROGS` in `Makefile`. Once you've done that `make qemu` will compile your program and you'll be able to run it from the xv6 shell.
- If the user forgets to pass an argument, `sleep` should print an error message.
- The command line argument is passed as a string. You can convert it to an integer using `atoi()` (see `user/ulib.c`).
- You must use the system call `pause()` in your solution.
- See `kernel/sysproc.c` for the xv6 kernel code that implements the `pause()` system call (look for `sys_pause`). `user/user.h` for the C definition of `pause()` callable from a user program, and `user/usys.S` for the assembler code that jumps from the user code into the kernel for `pause()`.
- Look at Kernighan and Ritchie's book *The C Programming Language (second edition)* (K&R) to learn about C.

Run the program from the xv6 shell:

```
$ make qemu
...
init: starting sh
$ sleep 10
(nothing happens for a little while)
$
```

Your program should pause when run as shown above. Run `make grade` in your command line (outside of qemu) to see if you pass the sleep tests.

Note: that `make grade` runs all tests, including for the tasks below. If you want to run the grade tests for one task type:

```
$ ./grade-lab-util sleep
```

This will run the grade tests that match "sleep". Or you can type

```
$ make GRADEFLAGS=sleep grade
```

which does the same.

This was the first task for this first assignment where you have some work that can be graded. You are required to make a commit and push it for each task as you complete them. Make sure you make a commit for your Task 02 work after you have tested it, and push it up to your student GitHub classroom repository.

Task 03: `sixfive` (moderate)

In this exercise you'll use the system calls `open()` and `read()`, C strings, and processing text files in C.

For each input file, `sixfive` must print all the numbers in the file that are multiples of 5 or 6. Numbers are a sequence of decimal digits separated by characters in the string `" -\r\t\n./,"` (note that there is a blank space in there among the other delimiters). Thus, for the six in "xv6" `sixfive` shouldn't print 6, but for example "/6," it should.

The following example illustrates `sixfive`'s behavior:

```
$ sixfive sixfive.txt
5
100
18
6
```

Some hints:

- You should add your file named `user/sixfive.c` for this task, and don't forget to update the `UPROGS` in the `Makefile`
- Read the input file a character at a time.
- You can test if a character matches any of the separators using the `strchr()` function (see `user/ulib.c`).
- Start and end of file are implicit separators.

Task 04: `memdump` (moderate)

This exercise will giv you more practice using C pointers. Before starting read section 5.1 (Pointers and addresses) through 5.6 (Pointer arrays) and 6.4 (pointers to structures) in *The C Programming Language (second edition)* by Kernighan and Ritchie (K&R).

Have a look at `user/memdump.c`. Your job is to implement the function `memdump(char *fmt, char *data, int len)`. `memdump()`'s purpose is to print the contents of the `len` valid bytes of memory pointed to by `data` in the format described by the `fmt` argument. The format is a C string. Each character of the string indicates how to print successive parts of the data. Thus for example, a C struct with multiple fields can be printed with a format string containing multiple characters.

Your `memdump()` should handle the following format characters:

- `i`: print the next 4 bytes of data as a 32-bit integer in decimal.
- `p`: print the next 8 bytes of data as a 64-bit integer in hex.
- `h`: print the next 2 bytes of the data as a 16-bit integer, in decimal.
- `c`: print the next 1 byte of the data as an 8-bit ASCII character.
- `s`: the next 8 bytes of the data contain a 64-bit pointer to a C string; print the string.
- `S`: the rest of the data contains the bytes of a null-terminated C string; print the string.

`memdump()` must respect the `len` argument and never read past the `len` valid bytes of data. If a format character would require more bytes than remain (for example an `i` that needs 4 bytes when only 2 are left), `memdump()` must print

`memdump: not enough data for `X``

where `X` is that format character that fails, and then stop. For the `S` format, print the string up to its terminating null byte or the end of the valid data, whichever comes first.

You will need to use C's `printf()` in your `memdump()`.

The `memdump` program, if executed with no arguments, calls your `memdump()` function with some example format strings and data. If `memdump()` is correctly implemented, the output will be:

```
$ memdump
Example 1:
61810
2025
Example 2:
```

```
a string
Example 3:
another
Example 4:
0CE8
1819438967
100
z
xyzzy
Example 5:
hello
w
o
r
l
d
```

You will likely get a different hex address for the first line of the Example 4 output.

If the `memdump` program is invoked with an argument, it will read its standard input up to an end of file, and then call `memdump()` with the format and input data. So, once `memdump()` is implemented:

```
$ echo deadc0de | memdump hhcccc
25956
25697
c
0
d
e
$ echo deadc0de | memdump p
6564306364616564
```

Note: In the second example the result is the hex values of the ascii characters `deadc0de` in reverse, which is expected because of the little endian layout of RISCV numbers in memory. However you will find that `"%lx"` to display a hex value in `printf()` seems to only format 4 bytes (32 bits), even using the `l` flag to say you are trying to format a long 8 byte (64 bit) number. In short, you will need to call `printf()` twice for the `p` formatter to correctly display all 8 bytes of the data.

The bounds check fires when the input is too short for a format character. Here `echo` supplies only two bytes ('a' and a newline), but `i` needs four:

```
$ echo a | memdump i
memdump: not enough data for 'i'
```

Task 05: `find` (moderate)

This exercise explores pathnames and directories, and the system calls `open()`, `read()` and `fstat()`.

Write a simple version of the UNIX `find` program for `xv6`. Find all the files in a directory tree with a specific name. Your solution should be in the file `user/find.c`.

Some hints:

- Look at and study `usr/ls.c` to see how to read a directory. A lot of your solution will reuse this structure to create a path and `stat()` it to determine if it is a directory that needs to be recursed into.
- Use recursion to allow `find` to descend into sub-directories
- Don't recurse into "." (current directory) and ".." (parent directory).
- Each time you invoke `make qemu` it will build a new fs.img, removing files created in previous run. This may slow you down here. If you would like to start qemu with the file system from a previous run, use `make qemu-fs`.
- You need to use C strings. Have a look at K&$ (the C book), for example section 5.5.
- Note that `==` does not compare strings as in Python. Use `strcmp()` instead. See `ulib.c`.

- Don't forget to add the program to `UPROGS` in Makefile.

Your solution should produce the following output (when the file system contains files `b`, `a/b`, and `a/aa/b`):

```
$ make qemu
...
init: starting sh
$ echo > b
$ mkdir a
$ echo > a/b
$ mkdir a/aa
$ echo > a/aa/b
$ find . b
./b
./a/b
./a/aa/b
```

Task 06: `exec` flag for find (moderate)

This exercise involves the system calls `fork()`, `exec()` and `wait()`.

Add a `"-exec cmd"` to `find`, which executes the program `"cmd file"` for each file `f` that `find` finds, instead of printing matching file names.

The following example illustrates `find -exec` behavior:

```
$ find . wc -exec echo hi
hi ./wc
```

Note that the command here is `"echo hi"` and the file that is found is `"./wc"`, making the command that should be `exec()`ed `"echo hi ./wc"`, which outputs `"hi ./wc"`.

Some hints:

- Use `fork()` and `exec()` to invoke the command on each file that is found to match the search pattern. Use `wait()` in the parent to wait for the child to complete the command.
- `kernel/param.h` declares `MAXARG`, which may be useful if you need to declare an `argv` array to call `exec()` with.

To test your solution for `find`, run the shell script `findtest.sh`. Your solution should produce the following output:

```
$ sh < findtest.sh
$ $ $ $ $ $ hello
hello
hello
```

The output has many `$` because the xv6 shell doesn't realize it is processing commands from a file instead of from the console, and prints a `$` for each command in the file.

Submit the Assignment

Time spent

Create a new file, `time.txt`, and put in a single integer, the number of hours you spent on the lab. Make sure you add and commit a final commit with this file as it is part of the autograder score.

Assignment submissions are handled by the GitHub classroom autograder. You should create at least 1 commit for each task as you do it, and push it to your GitHub classroom. You can check that the autograder tests are passing in GitHub classroom by looking at your submitted releases. The tests run in the GitHub classroom are the same ones that you can run locally by using `make grade` or `./grade-lab-util test`.

- Please run `make grade` to ensure that your code passes all of the tests. The GitHub classroom autograder will use the same grading program to assign you an autograder evaluation score for your work.

Optional Challenge Exercises

- Write an `uptime` user program that prints the uptime in terms of `ticks` using the `uptime()` system call. (easy)
- Support regular expressions in name matching for `find`. `grep.c` has some primitive support for regular expressions. (easy)
- The xv6 shell (user/sh.c) is just another user program. It lacks many features found in real shells, but you can modify and improve it. For example
  - Modify the shell to not print a `$` when processing shell commands from a file (moderate)
  - Modify the shell to support wait (easy)
  - Modify the shell to support tab completion (moderate)
  - Modify the shell to keep a history of passed shell commands (moderate)
  - Or anything else you would like your shell to do. If you are very ambitious, you may have to modify the kernel to support kernel features you need; xv6 doesn't support much out of the box.